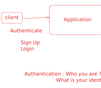
Authentication & Authorization



Authentication : Who you are ?
What is your identity?

Authorization : What you can do ?
Access control / access policies.

Authenticate : Login (username & passwords)

Authorization : Access rights.

Requirements :

1. User Registration : Sign up : Allow users to register by providing necessary information.
2. Login : Authenticate users based on their credentials.
3. Multi Factor Authentication (MFA) Implement a robust MFA system.
4. Password Recovery : Provide a secure process for users to recover there passwords.
5. Session Management : Efficiently manages sessions of the users to ensure security. In a state-less architecture we need to manage sessions of the users.
6. Access Control : To manage user roles (Admin, user etc) to see what they can access.

Back of the envelope calculations:
User Traffic : 100K users per day.

QPS : 100000/86400 = 1.15 req/sec

Storage Unit :

We would need to store the details of the users for authentication & authorization
Assume : 5KB / user

Estimating for 5 years.

Total storage = 100000 * 365 * 5 * 2 *10 * 5
= 0.1 * 10^5 * 365 * 5 * 5 * 2 *10
= 914 * 2 *10 * 2 *10
= 914 * 4* *10
= ~ 914 GB

Session Management
Stateless Architecture

Session Ids (Session based Auth)
vs
Token Ids (Token based Auth) --> JWT

HTTP / HTTPS
Headers : sessionId

Session --> connection

Db : Table --> Session table
user sessionId
TTL (15 mins)

Token based Authentication--> JWT

More secured way.

Tokens :

Types of tokens :
1. Access token
2. Refresh token

1. Every Req after authentication will come with an access token.
2. Now our application server will decrypt the access token. (using its own secret key).
3. Checks the TTL of access token.
4. If not expired, application server will check user permission.
5. Then user is allowed to interact.

Refresh token :
Access token --> TTL (Time to live)
Time --> short time (1 hour)

Refresh token --> TTL (Long --> 1 day)

Every Req after authentication will come with an access token.
A huge data set of commonly used passwords.
Now our application server will decrypt the access token. (using its own secret key).
Checks the TTL of access token.
If access token is expired, User will generate a new token.

Tokens : JWT tokens.

GET /user/profile
Authorization : Bearer eY12.3ad8121ad8

Secret key :
token --> encrypt
encrypted token. --> decrypted token (secret key)

Q: Why do we even need 2 tokens ?
Access token --> TTL (Long)

Access token --> will be sent with each HTTP request.
Hacker --> access token (it will have limited time to exploit)

Refresh token --> never sent to API calls. Only when new token needs to be generated.

Gmail : Web browser
login in Gmail --> access token valid for 1 hour.
You have your browser unatthended (3 hours)
Browser (auto refresh)
come back -->gmail silently uses refresh token to get a new access token.

after few days : refresh token is expired.
Gmail --> sign in again.

How passwords are stored in your APPlication DB?
1st way : store plain text.

```
login
{
    username : "aditya"
    password : "adtya123"
}
```

Store hashed passwords.

Storing Passwords Hashes.

adtya123 --> hash(adtya123) --> qwe1234 --> save to DB

What is Rainbow table ??
A huge data set of commonly used passwords.

SHA-1, SHA-256, Bcrypt, Argon2
password --> hashes(password)
password123
adtya123

permutation & combination (a-Z 0-9 a-z)

3 way to store passwords :
Salt & Pepper technique.

user --> adtya
password --> adtya123
salt : random string (tyu78) = adtya123 + tyu78
hash(adtya123tyu78) = wer3247
Store the salted and hashed password in DB.

hashpassword salt
wer3247 tyu78

pepper : What is pepper ?
pre defined string : asdf123 (at a very very secured location)
adtya123 --> adtya123tyu78 --> asdf123adtya123tyu78

hash(asdf123adtya123tyu78) =. were324
hashpassword Salt
were324. tyu78

plain password --> plain password + salt + pepper --> hash.
store hash to DB.

Good hash functions ?
Password --> hash(password)
Hash function should not fast.
basic hash function --> SHA-1, SHA-256 , MD5 etc
they can hash billions of records in a second.

Rainbow attack
password --> hash(password) --> (micro seconds)

Why Slow hash functions are good ??
Slow hash functions make it infeasible
for a attacker to have rainbow attack.

Good hashing algo --> Argon2, bcrypt, scrypt etc
Slower (milli seconds)
user experience will not be impacted
but hacker experience will be impacted.

brute force attack --> infeasible.

Authorization : What can you do? Access rights
1. Role based access control (RBAC)
2. Policy based access control (PBAC)
3. Access control List (ACL)

RBAC --> Roles (Admin, member, owner etc)
Admin --> kick a member out
Memeber -->
Owner -->

PBAC : Define dynamic rules / privileges --> policies
Policy --> Some user can access paid Questions
nee policy : No more 3 requst per second.

ACL : Highly granular control
Individual permissions :
Fun onoff.
For every user diff permissions can be on/off.

API End points:
POST www.instaramclone.com/register

```
{
    "username" : "aditya",
    "fullname" : "",
    "password" : "",
    "mobile/tel" : "",
    "email" : ""
}
```

```
/login
{
    "username" : "",
    "password" : ""
}
```

```
/logout
```

Mark user session as inactive. (Revoke authentication).

Others :
/create/{user}
/grant/{user}/{permission}
/revoke/{user}/{permission}
/expire/{session}
/validate/{session}

Table Structure:
User Table : to store basic user info
userId, email, name, username, role, createdAt, updatedAt etc.

Credential table:
credentialID, emailFK), access_token, refresh_token

password Table:
passwordId, emailFK), hashed_Password, salt, last_login etc.

Session table : Active sessions of user (history)
sessionId, emailFK), last_logged_in etc.

User Registration:

Login user

Session Management

Complete Flow

Logout user: